

→ mids exclusive

Solve handout problem (iteration)

Date: _____

layer 2 & more from handout. dry run

→ We cannot find derivative directly but with chain rule.

Mar 25, 2026

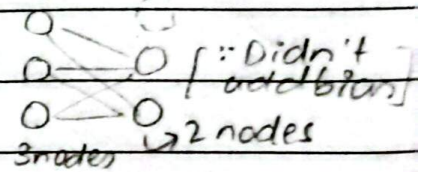
→ We are using stochastic gd as optimizer for simplicity.

Dimension of vector : 3×2 : 3 nodes each $\times$ 2 layers (layer 1)
(of weights) 3×1 layer 2

→ Bias can be added or not.

→ On stochastic, we have to update weights after average.

→ layer number of next layer is superscript



BACKWARD PROPAGATION:

→ $J(w)$ and the layer is only differentiated easily

The weights of layer 2 can easily be used with $d(Jw)$ but with layer 1 there are multiple paths and chain rule is used.

- On the end, the sum of paths is taken.

→ Error is not calculated for Bias.

→ Output layer has fixed formula for error calculation

$$\delta_1^n = \hat{y} - y$$

Element-wise product

→ Chain rule error calculating via path.

current node derivative $\cdot$ weight $\cdot$ layer's error of next error.

↳ path error.

→ Next layer error yields prev. layer error.

→ This will now be taken as chain derivative.

$$\delta_1 = \delta_2 \cdot w_{12}$$

chain derivative = Current node $\cdot$ path error (forward)

mids

ISSUES IN ARTIFICIAL NETWORK

L2 is by default (mostly) and is also popular.

DROPOUTS: To reduce Overfit
Fully connected / Dense Layer:

→ No dropout can be applied on o/p layer.

→ Randomly turn off neurons at each iteration to simplify the network for forward or backward propagation.

Date: _____

→ Dropout Ratio (p) : hyperparameter

- to decide ~~rate~~ how many dropout ratio for each layer.

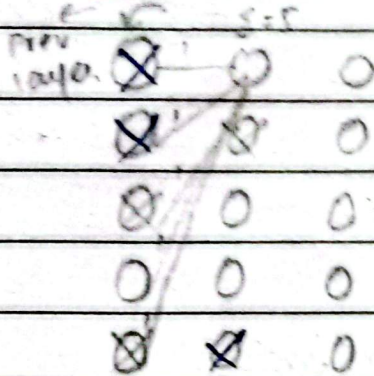
- Can be ~~different~~ differently on each layer.

- Not necessarily applied on each layer.

- Range (0-1) : probabilities.

- 0.4 : 40% neurons of layer turned off at layer k.

- One neuron can be selected more than once.



- This is done during training.

- Drawback In Testing:

- Actual $S=5$ but after dropout $S=3$.

But when we apply testing or $\%$

in real world, dropout is not

applied so the $S=5$ instead of $S=3$.

Test Data → No dropout.

Train Data → Dropout.

0.4 0.2 0 (zero)

Solution:

(1-p) hyperparameter : keep Prob.

is introduced. So that either,

after dropout (1-p) is multiplied to test or divide train by (1-p)

X : iter 0

X : iter 1

→ Scale up test = $(1-p) \times S$

→ Scale down train = $S/(1-p)$

→ We cannot risk or p layer, so no dropout applied.

Keras : API to build network.

Tensor Flow may use Keras.

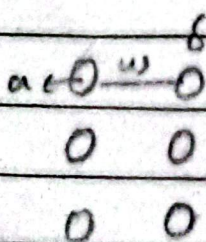
Name to maintain consistency in code.

Drop out is applied to prev. layer.

THE VANISHING EXPLORING GRADIENT PROBLEM:

- if any of the 3 operand small, result smaller (Vanishing gradient)

- if any of the 3 operand large, result large (Exploring Grad. issue)



→ sigmoid = saturating activation function (check for tanh as well)

VANISHING/EXPLODING GRADIENT PROBLEM

- max der. value in sigmoid is 0.25 and we need a derivative in error equation. So, over multiple layers the error equation product $\delta^{(L+1)} w^{(L+1)}_i a'_i$ will get smaller and smaller and then the weights at the very backstaging layers will get stuck and will not update while others will. that no update very small
- by which gradient becomes almost zero
Ans. → till reaching the start, the error & der eq. will become only starting weights not being updated. Over very slow iteration (How to identify vanishing gradient?)
- Saturating activation functions are a problem.
- Another issue is weight initialization.
- Variance is distribution.
(fan in $\neq$ fan out)
- L2: each node will sum the 2 nodes but L3 will sum the 4 nodes which is greater than L2. This is an issue. (Another problem, soln: weight initialization technique)

WEIGHT INITIALIZATION TECHNIQUE:

- We cannot assign same value to all weights, since every node is working on different aspect, needs different value. also, if same, every node will yield same value.
- Gaussian/Normal.
bias are initialized with zero
- The tail is asymptotic and the values most around mean. since, asymptotic, the value may exceed 1.
- Use standard normal for weights: $SN(0,1)$ → check
- Uniform distribution:
→ use for bias weights

Soln:

- weight initialization should be proportional to the fan in & fanout of that layer. This means that scale (variance) of initial weights should depend on how many connections a neuron must have both incoming & outgoing.
- Exploding gradient means: happens when forward propagation the gradient becomes very large during training.

$$\text{default: weight} = \sqrt{\frac{16}{f_{in} + f_{out}}} \quad \sqrt{\frac{16}{f_{in} + f_{out}}}$$

Unbounded value = Not Saturating Function

Date: _____

→ solution of exploding gradient.

New Handout

BATCH NORMALIZATION

→ hidden layers:

→ pre normalize: tanh & sigmoid normalize prefer this.
+ Normalize z-values. and apply of afterwards.

sequential → post normalize:
+ Apply normalization after of.

pre → Input(dim) → if pre-normalize.
Dense (specify units) output of one line is automatically
↓ paired to the next line

Fully connected layers.

Batch Normalization()

Activation ('tanh')

post → Dense (128, 'relu'),
Batch Normalization(),
Dropout(0.5)]

Handout 12

SPECIALIZED NN:

Flattening of Image Data:

Default image dataset storage: 4D

→ we have to open the 4D for NN.

→ So, we convert the 4D to 2D.

Known as **Flattening of Image Data**.

→ After this → normalization → split

image (2) {
Image with four pixels.
row
color
[[[r, g, b, a], [r, g, b, a]],
[[r, g, b, a], [r, g, b, a]] }

image (2)

color depth is 3d dimension

SHORTCOMING OF ANN:

1) SPATIAL INFORMATION LOSS:

reason: flattening of array.

soln: enhancement: CNN (convolutional NN) can take 3D.

2) POOR SCALING:

By adding one neuron, the weight increases exponentially.
20x20 → 25x25 (eg).

reason: full connectivity. / dense
enhancement: CNN

3) Cannot handle sequential data.

cat eat or cat eats depends on cat, sequential data
enhancement: RNN, recurrent NN.

TRANSFER LEARNING:

Use already made model by transfer learning

What is transferred?

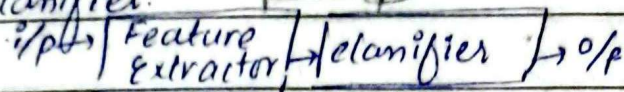
→ Architecture → weights

Why it works:

→ Images have some basic features.
→ So, if the layers trained for basic, can be transferred. We freeze them, hence, the weights become **undrainable**.

3-ways:

1) Use pre-trained NN as classifier.

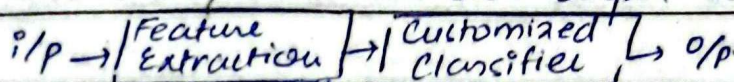


Target: Dog
Source: Dog
Eg

source domain: Data on which model already trained
target domain: Data on which model will be trained.

→ If source domain = target domain; no extra work needed.
→ 10,000 Image Net → 10,000 classes → 10,000 output neurons.
→ VGG16 & Inception Resnet v2 model.

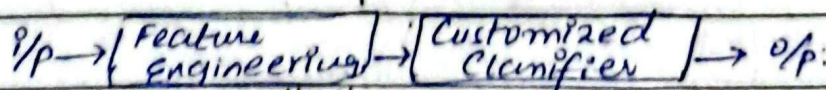
2) When we want our own output classes.



→ source of target & domain are somewhat similar.
→ New customized classifier & training required.

3) Fine tuning

When we want our own extracted features.



freeze (trained on our requirement) → train

target
→ pretrained must have already fetched you there

NEURAL CONVOLUTIONAL NETWORK

Date: Apr 13, 2026.

→ Also called ConvNets.

ANN:

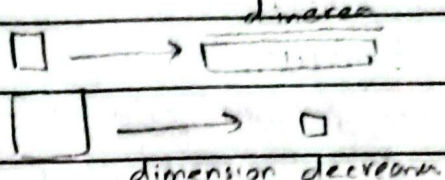
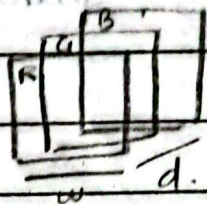
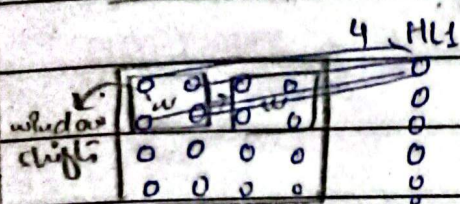
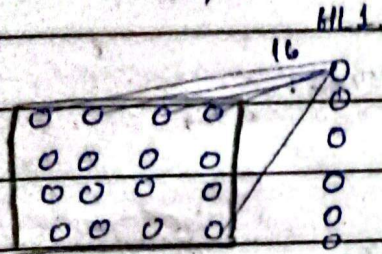
$4 \times 4 \Rightarrow 16$ inputs.

Vanilla Neural Network:

Simplist Form of ANN:

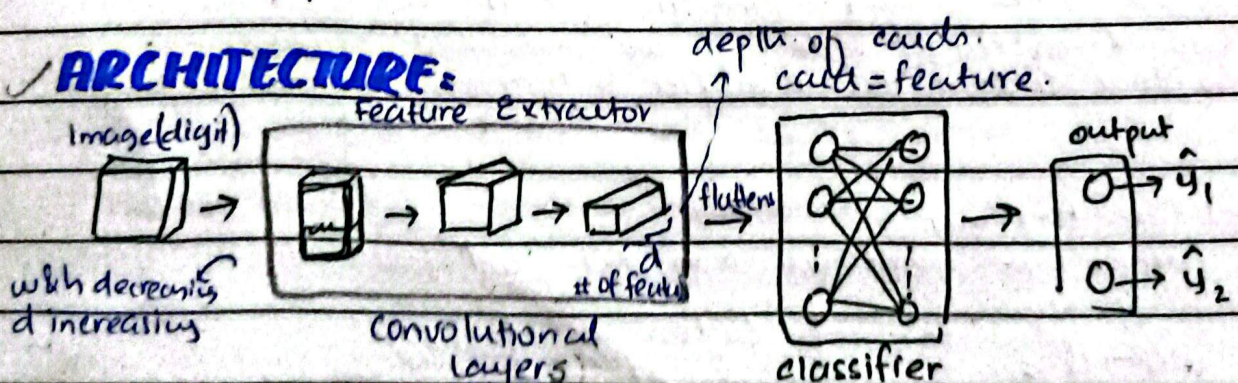
convolutional layer scan be used

ANN cannot be used



- In CNN, due to window, the size of input reduced.
- CNN is not fully connected whereas ANN is.
- Number of When image is scaled up, the window size remains same in CNN, hence structure remains same & weights don't increase exponentially as in ANN. Improving scaling problem. as here, we have taken window size: 4 inputs.

ARCHITECTURE:



- No flattening, dimensions maintained. In convolutional layer
- On classifier, fully connected layers. Similar to ANN
- They some time have greater # of weights than convolutionary layer.

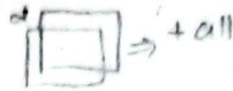
CONVOLUTION LAYER:

→ CONVOLUTION OPERATION:

→ We can have max image
 $n_h \times n_w$

→ dm: depletion

Feature Map = Activation map



Date: _____

Example 1:

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

convolution operation

1	0	4
1	0	4
1	0	7

Filter/

Kernel
(weights)

 3×3 ~~fff~~

Feature Map

$$(n-f+1) \times (n-f+1)$$

→ Height & width reduces
Dimension reduces.

→ ^{Each} Feature map extracts one feature.

- Feature map extracts one feature.
- Kernel is initialized, diff. initialization for diff. feature weights are for vertical lines.

→ Suppose, we initialized weights are for vertical lines.

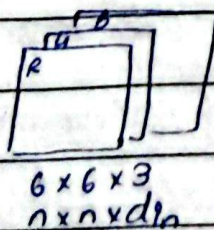
→ For this, filter is magnifying glass that will start from top-left, slide to each cell, relevant cell value in feature map = 1 if vertical line else 0. Then move to next row. This sliding causes a lot of computation.

weighted sum:

Weighted sum:
 $3 + 0 + (-1) + 1 + 0 + (-8) + 2 + 0 + (-2) \Rightarrow -5$ Exactly same for each window.

Example 2:

Example 2:
convolution over volumes.


$$\begin{array}{l} 3 \times 3 \times 3 \\ f \times f \times d_{in} \\ (27 \text{ inputs}) \end{array}$$
 ~~$(n-f+1) \times (n-f+1)$~~

sum of 27 products
" 27 inputs

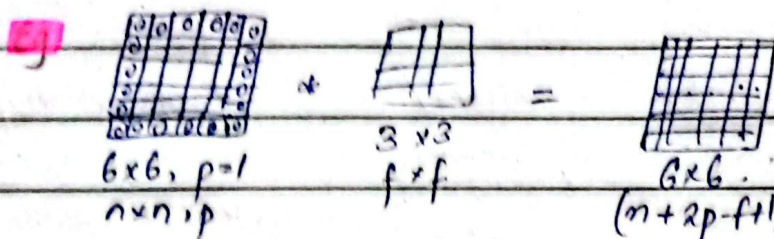
→ Dimension of filter must be equal to dimension of kernel :
placing over volume.

→ dia does not effect the douf.

HYPERPARAMETRES:

1. PADDING: (P)

The dimension reduces after convolution. If we want to preserve the dimension, padding is used; this adds a layer to original image.



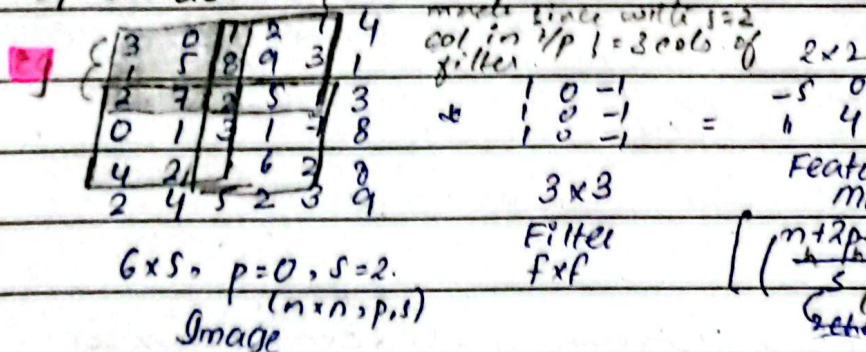
$$\begin{matrix} 6 \times 6, p=1 \\ n \times n, p \end{matrix} * \begin{matrix} 3 \times 3 \\ f \times f \end{matrix} = \begin{matrix} 6 \times 6 \\ (n+2p-f+1) \times (n+2p-f+1) \end{matrix}$$

Python:

- Valid: same no padding size
- Same: FM has same size as 1/p

2) Stride: (s)

Another window cannot be made since with $s=2$



$$\begin{matrix} 6 \times 5, p=0, s=2 \\ (n \times n, p, s) \\ \text{Image} \end{matrix} * \begin{matrix} 3 \times 3 \\ \text{Filter} \\ f \times f \end{matrix} = \begin{matrix} 2 \times 2 \\ \text{Feature Map} \end{matrix}$$

apply floor, round down, points involved.

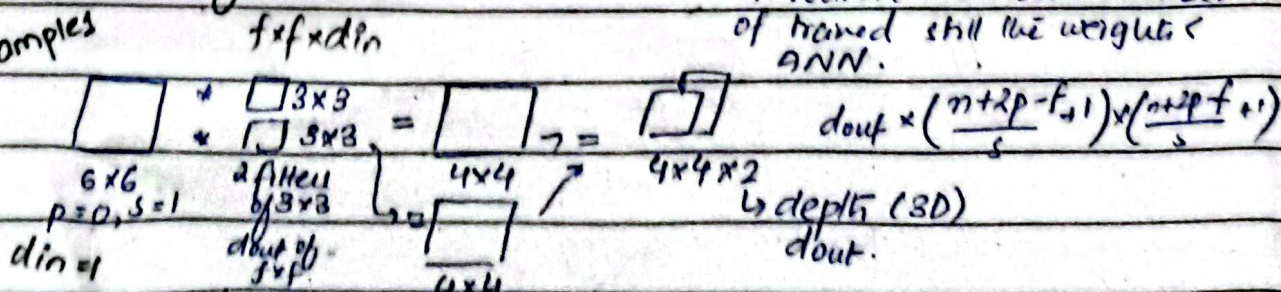
$$\left(\frac{n+2p-f}{s} \right) + 1 \times \left(\frac{n+2p-f}{s} \right) + 1$$

- Sliding causes a lot of computation. So we introduce stride.
- So, we slide till one column of previous slide included. But, the last column remains unvisited. Here, we have 4 sliding while by traditional approach, 16 slides.

3) Number of Filters:

Number of filters & Numbers of feature trained & Number of trained still the weights & ANN.

Examples



$$\begin{matrix} 6 \times 6 \\ p=0, s=1 \\ \text{din} = 1 \end{matrix} * \begin{matrix} 3 \times 3 \\ 2 \text{ filters} \\ 3 \times 3 \\ \text{depth of } f \times f \end{matrix} = \begin{matrix} 4 \times 4 \\ \text{dout} \end{matrix} \Rightarrow \begin{matrix} 4 \times 4 \times 2 \\ \text{depth (SD)} \\ \text{dout} \end{matrix}$$

- independent convolution of each filter.
- Number of filters = depth of the outp

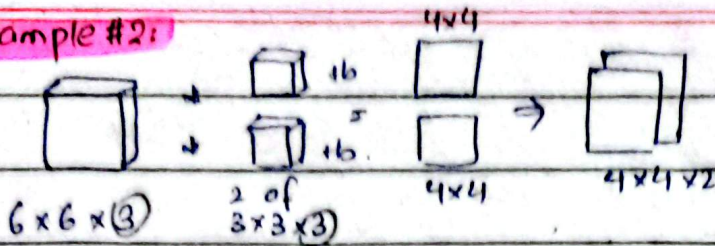
Every filter has its own depth.

$$\text{din} = \text{dfilter} \quad \# \text{ of filter} \Rightarrow \text{dout}$$

Number of Bias = Number of Filter

Date: _____

Example #2:



more the filters, more good the model.

$$\Rightarrow \text{of } (\boxed{} + \text{bias}) \Rightarrow \boxed{}$$

1 per filter.
But too
Just like 1 filter = 1 hidden layer.

4) Filter Size

→ Common Size: 2x2, 3x3

Small size: high precision

Filter Size

big size = skips the details but less computat.

$$\# \text{ of parameters} = (\text{filter size } (f_w \times f_h \times f_d)) \times \# \text{ of filter}$$

+ # of filter (bias)

→ Weights are inside filter.

$$(3 \times 3 \times 3) \times 2 + 2 \Rightarrow 56$$

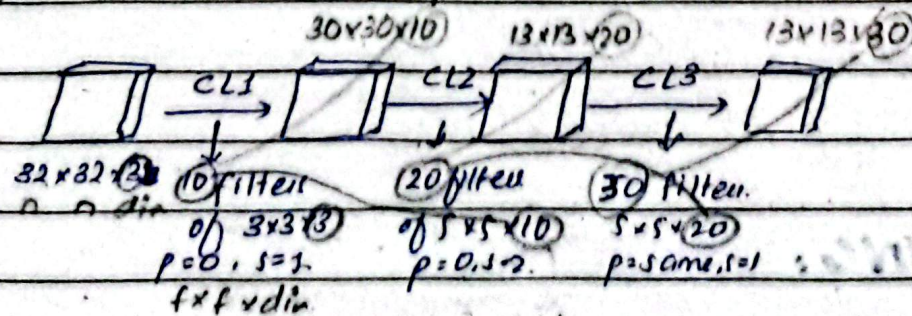
Exercises:

1) Find # of weights.

Input	Filter	# of parameters	dim # of filter
6x6	3x3	9 + 1 = 10	(3x3x1x1) + 1
8x8	3x3	9 + 1 = 10	↑ weights same, depends on filter
6x6	2 of 3x3	(3x3x2) + 2 = 20	
6x6x3	2 of 3x3x3	(3x3x3x2) + 2 =	
6x6x3	10 of 3x3x3	(3x3x3x10) + 2 =	

2) Cascaded Convolutional layers.

af: if math.



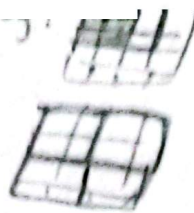
high
low

First 2 dim by formula, last = No. of filter.

CNN layer Pooling

- learnable weight
- Pooling can be an.
- No learnable wgt
- Does not use padding

non over



Date: April 16, 2020

Handout 13 Pooling Layer

- In max pooling, the strongest feature is extracted with the idea that rest are weak and are not necessary to keep.
- Filter creates depth, the pooling is applied to each filter. Filter size 'depth' will not be affected.
- Pooling layer is not a weight layer.
- Pooling layer is never trained while convolution is.
- Non Dimension reduction via CNN. So, modern architecture doesn't use pooling layer.

f = pooling size. $\text{larger } f$: Info. loss
∴ one will be chosen & a lot will be wasted

→ Cascading Convolutional and Pooling Layers

→ Classification is done on flattened layer.

20 Apr. 2021

RECURRENT NEURAL NETWORK (RNNs)

Sequence Data:

- Time based data
- Non-time based sequence data: audio, text

Examples:

Applications

Sentiment Analysis

Machine Translation

Name Tagging

Language Modelling

Input

There is nothing to like in this book

How are you?

I met Sara & Ali

I would like to have

like after nothing to make intra dependency this will decide whether a good word or not.

1-5 or

-ve/ve/neutral

مايكون شيئا

- word to word

I met Sara and Ali

I would like to have

tags that Sara and Ali

→ Models should learn the interdependency as well interdependency: dependency between two or more examples. Intra ∪ ∪ ∪: inside an example, e.g. audio, text.

→ Input/output pairs are needed for all above.

like -ve/ve for sentiment, word to word translation for machine translation etc.

→ The length of input is variable here.

TEXT ENCODING:

→ Texts are not acceptable to machine. So, encoding needed. Example: Cat eats Cheese.

= (encode('Cat'), encode('eats'), encode('cheese'))

↳ simple yet useful

Enhancement:

LSTM:

→ Long-short term memory.

→ Gated recurrent net

→ Transformer for long words.

Date: _____

Instead of word embedding, a better approach

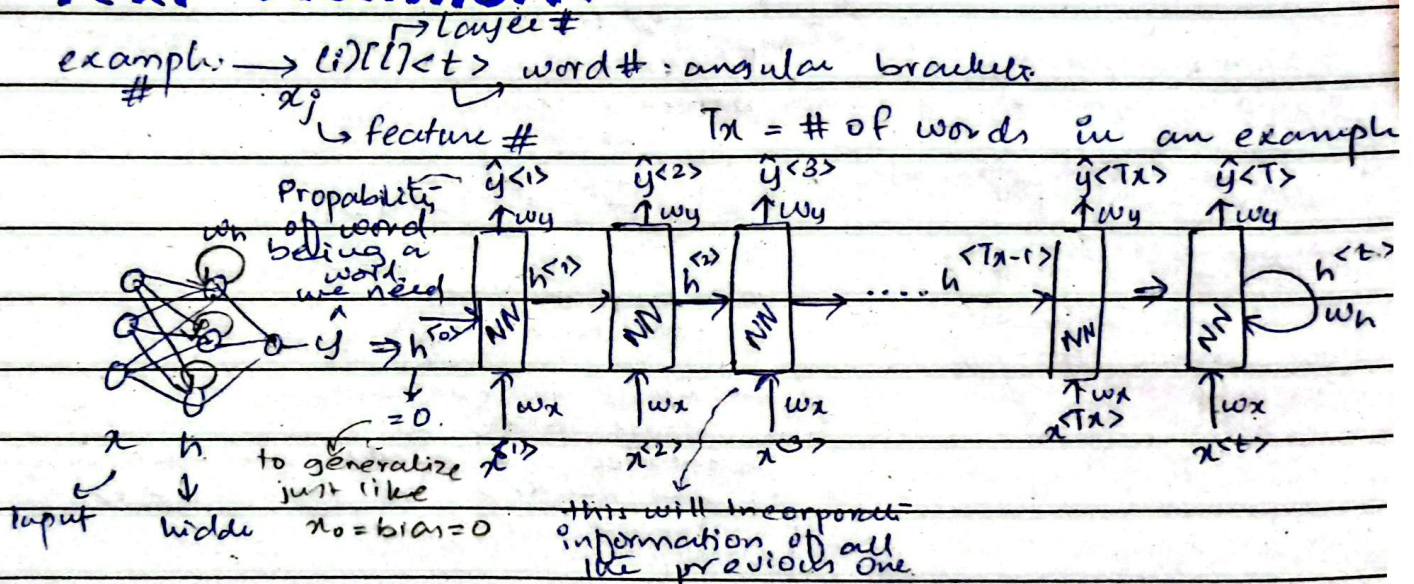
$$\text{word embedding} = \begin{bmatrix} \begin{bmatrix} \vdots \\ \vdots \end{bmatrix}, \begin{bmatrix} \vdots \\ \vdots \end{bmatrix}, \begin{bmatrix} \vdots \\ \vdots \end{bmatrix} \end{bmatrix}$$

4 4 4

Every word is vectorized including 4 characteristic that past, present etc. So this becomes 12 inputs. We cannot feed all features together since interdependence exist.

TEXT NOTATION:

example: $\rightarrow (i)[[t]<t>$ word #: angular brackets



$h^{(t)}$ variable has to be maintained that gets updated

$$h^{(t)} = af(x^{(t)}, h^{(t-1)}) \Rightarrow \hat{y} = af(h^{(t)})$$

→ Feedback connection too have weights known as w_h .

$$h^{(t)} = af(x^{(t)} w_x + h^{(t-1)} w_h + \text{bias})$$

→ The networks aren't deep: a lot of computation

Memory:

short term:

if small memory, the word cat can track that it cat and mouse cat, which is brown eats cheese = cat.

some architecture says, output of previous input, comes as input is not correct

Date: Apr 22, 2026.

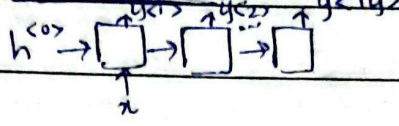
TYPES OF RNNs ON THE BASIS OF INPUT OUTPUT SEQUENCE:

One-to-One



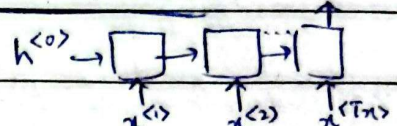
- ANN/CNN
- Eg: Image Classification

(vector to sequence) One to Many



- Input is a vector (single)
- Output is sequential.
- Eg: Image Captioning by AI

(sequence to vector) Many to One



- Classification vector in output
- Output is a sentence in a dependency word by word input.
- Eg: Sentiment Analysis.

Many to Many

Same length of i/p & o/p.



- Sequence to Sequence
- Eg: Name Tagging.
- $T_x = T_y$

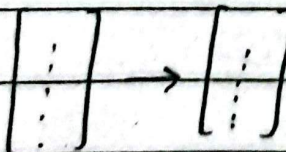
Different lengths of i/p & o/p.



- $T_x \neq T_y$ → Encoder
- $T_x \neq T_y$ → decoder
- Eg: Machine Translation
- $T_x > T_y$ or $T_x < T_y$
- T_x & T_y are not constant either in examples
- Encoder and Decoder are 2 RNNs and are needed because $T_x \neq T_y$ lengths not fixed.

EMBEDDINGS IN DEEP LEARNING:

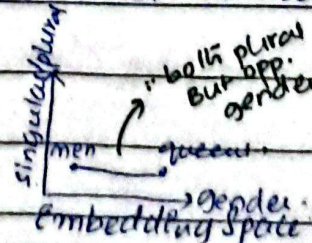
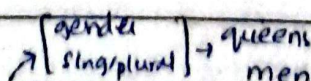
- Related to Generative Learning Somehow.
- Embedding maps high dimensional vector to a low dimensional vector.



Embedding

→ Intelligent Compression; know what features to extract eg: Background remover AI.

Two types:

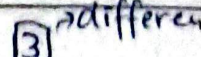
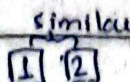


→ Word Embedding:

→ We can create a graph for these embeddings. If good embeddings related words will be near in embedding.

→ Visual Embedding:

img1, img2 → Miss Marie
img3 → Miss Zarein



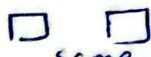
ONE SHORT LEARNING:

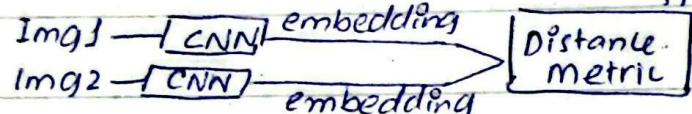
- Embedding can be used to create an encrypted form of someone's picture rather than storing 1000+ images for training. Now, new image can be detected by difference leucidea between embedding, and new image's embedding.
- Eg: Face recognition, image matching, object reidentification, finger print-signature matching.
- Where pixel by pixel matching fails.
 smallest difference, tanning, height, new capture, face capture etc.

Train:

Image $\rightarrow$ [CNN] $\rightarrow$ []

Dataset

x	y
 same	1
 diff	0

→ Siamese N/w: 

- If similar img, it will try to minimize the distance, if diff. img. it will try to maximize the distance.
- Once, trained, these CNN's can be used as embedded. The weights are updated in CNN.

Handout 16

Apr 23, 2026.

- Discriminative & Generative are opposite.

Latent Dimensional Vector:

- This lower dimensional vector is intelligent.
- Generative model should be able to create embeddings, and latent space.
- Latent distribution maybe of any distribution. Any value picked from this space & converted to original input.
- Averaging two diff. vector, a new image when converted containing the characteristics of both the images. This is the idea behind the generative idea.

Auto Encoder:

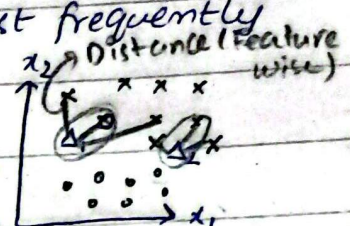
- Encoder is input, Decoder is output.
- We can add customization in encoder as well.
- The deeper the embedding, the stronger it is. Layer size decrease.
- x : Original input, $\hat{x}$: Generated.
- x & $\hat{x}$ are compared in last, & their diff. is minimized using MSE, euclidean etc. The weights are updated in each minimization loop in the embedding.
- Transpose Convolution = upsampling = Decoding.
- The image generation accuracy depends on trained weights.
- Images $\rightarrow$ CNN, text: to retain word embedding $\rightarrow$ RNN
- We can create word embedding for captioning.

K-NEAREST NEIGHBOURS (KNN)

April 28, 2021

KNN CLASSIFIER:

- Stores all training examples without any computation.
- When a new sample x arrives:
 - Identifies k nearest neighbours.
 - Identifies the class of which is most frequently found among the k -neighbours.
 - Assigns C to x .



DISTANCE METRICS:

1. EUCLIDEAN DISTANCE:

$A, B =$ 2 points
 $A = (a_1, a_2, \dots, a_m)$ ($m = \text{Feat}$, $A = \text{vector}$)
 $B = (b_1, b_2, \dots, b_m)$ ($L_2 \text{ Norm}$)
 $d(A, B) = \sqrt{\sum_{i=1}^m (a_i - b_i)^2}$ $\|a_i - b_i\|_2$

2. MANHATTAN DISTANCE: ($L_1 \text{ norm}$) $\|a_i - b_i\|_1$

$$d(A, B) = \sum_{i=1}^m |a_i - b_i|$$

3. MINKOWSKI DISTANCE:

→ General Formula
 $d(A, B) = \left(\sum_{i=1}^m (a_i - b_i)^p \right)^{1/p}$

This formula is known as $L_p \text{ norm}$.

General Repr: $\|a_i - b_i\|_p$

$p = \text{Order of metric}$

if $p=2$, Formula = Euclidean } special cases
 if $p=1$, " " = Manhattan. } of Minkowski

EXAMPLE:

Training Example	Feature vector	Class	Distance	Target
1	[1, 3, 1]	1	$\sqrt{(1-2)^2 + (3-4)^2 + (1-2)^2} = \sqrt{3}$	4
2	[3, 5, 2]	1	$\sqrt{2}$	6
3	[3, 2, 2]	0	$\sqrt{5}$	8
4	[5, 3, 2]	0	$\sqrt{14}$	10
x	[2, 4, 2]	?		

Continued x, y example

$k=3$

$\Delta_1 = 2 \times 1, 1 \times 0$

max $\rightarrow 2 \times$

$= x \text{ class}$

$\Delta_2 = 3 \times 1$

$\Delta_3 = x \text{ class}$

if $k=1 \Rightarrow x \rightarrow 1$ min dis of all $\therefore$ we have only one point
 $k=3 \Rightarrow x \rightarrow 1$ min dis of 3.

if KNN regressor.

$k=1 \Rightarrow x=4$

$k=3 \Rightarrow x = \frac{4+6+8}{3} \Rightarrow 6$

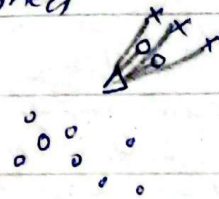
KNN-REGRESSOR:

- Finds k -nearest neighbour.
- Averages the target values of these k -neighbours.
- Assigns the average to x .

WEIGHTED NEAREST NEIGHBOURS:

- closest points should have higher weights
- For the given example, if we had assigned weights, high prob. that the sum of o 's $>$ sum of x 's w

$$w_i = \frac{1}{d(a, x_i)}$$



$k=5$, logically must be closest to o but due to majority being x , x wins.

classification → Higher combined weights of a class.
Regression → Find weighted average.

The larger the k , the better the result but not too large.
As large k beats outliers & kills noise.

HYPERPARAMETRES:

- Binary Class → k odd, no need of tie. → random selection: pp
- Multi Class → $k=7$ $C_1=3$ $C_2=3$ $C_3=1$
- multi-class needs tie breakers.
- The value of k is hyperpara.
- p is hypermb.

DATA PREPROCESSING:

- k -NN needs scaled data
- Should not contain irrelevant features, as their values & existence will effect distance formula.

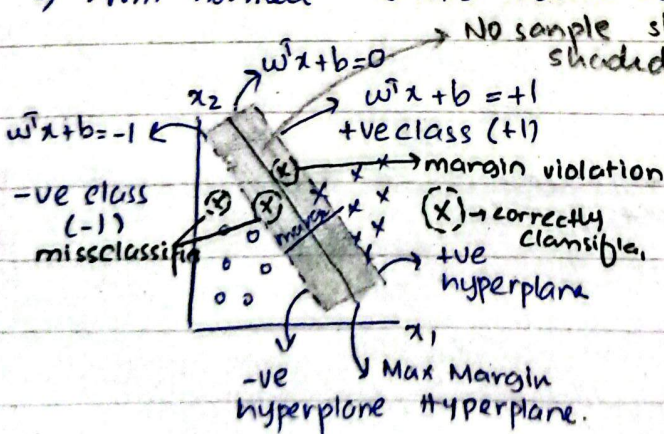
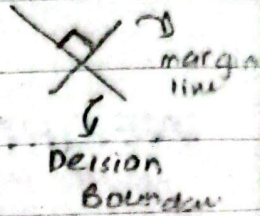
class $\rightarrow +1$
 -ve class $\rightarrow -1$ } always.

Handout 18

April 30, 2026

SUPPORT VECTOR MACHINE:

- $\rightarrow$ SVM finds the decision boundary is marginal.
- $\rightarrow$ Finds the decision boundary that gives maximum marginal.
- $\rightarrow$ marginal line is always perpendicular to the decision boundary.
- $\rightarrow$ makes super partition, includes buffer as well.
- $\rightarrow$ We call it hyperplane since, greater dimensions: for n features, $n-1$ dimensional feature line. line $\rightarrow$ Plane $\rightarrow$ HyperPlane.
- $\rightarrow$ SVM is non-probabilistic.
- $\rightarrow$ Non-easily classifying data: non-linear boundary.
- $\rightarrow$ Points, below neg. hyperplane, ...
- $\rightarrow$ Points not allowed to enter the band.
- $\rightarrow$ Once hyperplane added, the decision boundary remains fixed even after adding new samples. Not like logistic regression which created a threshold.
- $\rightarrow$ Two objectives: 1) maximize marginal line (keep w small)
 2) Samples must not cross border neither margin needs heavily penalized
- $\rightarrow$ Support hyperparameters eq: $w^T x + b = 1$, $w^T x + b = -1$
- $\rightarrow$ Outside marginal: 0 loss, inside marginal = some loss, Outside marginal: Heavily penalized.
- $\rightarrow$ Form normed $w^T x + b$ should be len than 1.

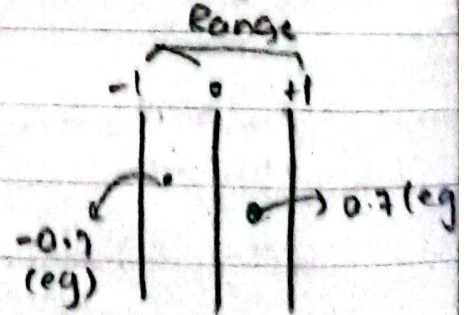


May 4, 2026

OBJECTIVE FUNCTION: weight vector

$$J(w, b) = \text{minimize } \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \max[0, 1 - y^{(i)}(w^T x^{(i)} + b)]$$

Maximize width of Margin by Minimizing the weight vector
 weight $\propto \frac{1}{\text{width}}$



If stochastic $\rightarrow$ Remove hinge loss from the eq.

$$[1 - 1(1.5)] = -0.5(-ve), \text{ chosen } = 0 \text{ value}$$

$$[1 - 1(0.5)] = 0.5(+ve) = \text{chosen } 0.5$$

$$[1 - 1(-1.5)] = 2.5(+ve) = 2.5$$

This is both
Gradient Descent.

→ For hard classifying → width need to be shrink less
we drop this idea of hard classifying also it can
lead to overfitting.
So, hyperparameter (c) is used.

Hyperparameter (c) :

small $c \rightarrow$ wider margin, more margin violations.

large $c \rightarrow$ narrow margin, lesser margin violations.

$c \rightarrow \infty \rightarrow$ Hard Margin Classifier.

$$\gamma = \frac{1}{c}$$

Absent